Evan Baesler
October 17, 2026
31151619

EE Design 1 Technical Report
Microcontroller SPI & DAC

Introduction

In this module, we touch on the implementation of communication protocols and DAC systems to create a waveform generator with four functions and variable output between 10-100Hz and 0-5V. As the Raspberry Pi Pico 2 does not have an on-board DAC system, we utilize an external DAC with SPI.

Methods

Our system utilizes two switches for waveform selection; implementing a 4-1 mux, which allows for selection between: 00 – Sine, 01 – Square, 10 – Triangle, and 11 – Sawtooth. For output control we use two potentiometers hooked up to ADCs for easy reading amplitude and frequency settings. This is preferable to direct connection as the frequency control requires internal logic, and it allows for more accessible routing with less overlapping wires.

Upon boot, the uploaded firmware generates tables of a variable amount of samples for each waveform through an easily editable function, these function as our digital values to send to the DAC to form a waveform type. The frequency control adjusts the period of a sample, and the amplitude adjusts the multiplier for the sample.

The DAC requires four bits for the control, and 10 bits for the voltage level (where the control is shifted to the left by 12 bits, and the voltage level by 2 bits, where the 2 LSBs are unused.) The script utilizes DACA, but is easily changeable to DACB through changing the control term. The script makes a two-byte buffer to help the user easily send both bytes back-to-back to the DAC.

When a period of a waveform ends, the script then samples the new amplitude and frequency, waveform select, and updates the output table upon starting the next waveform period.

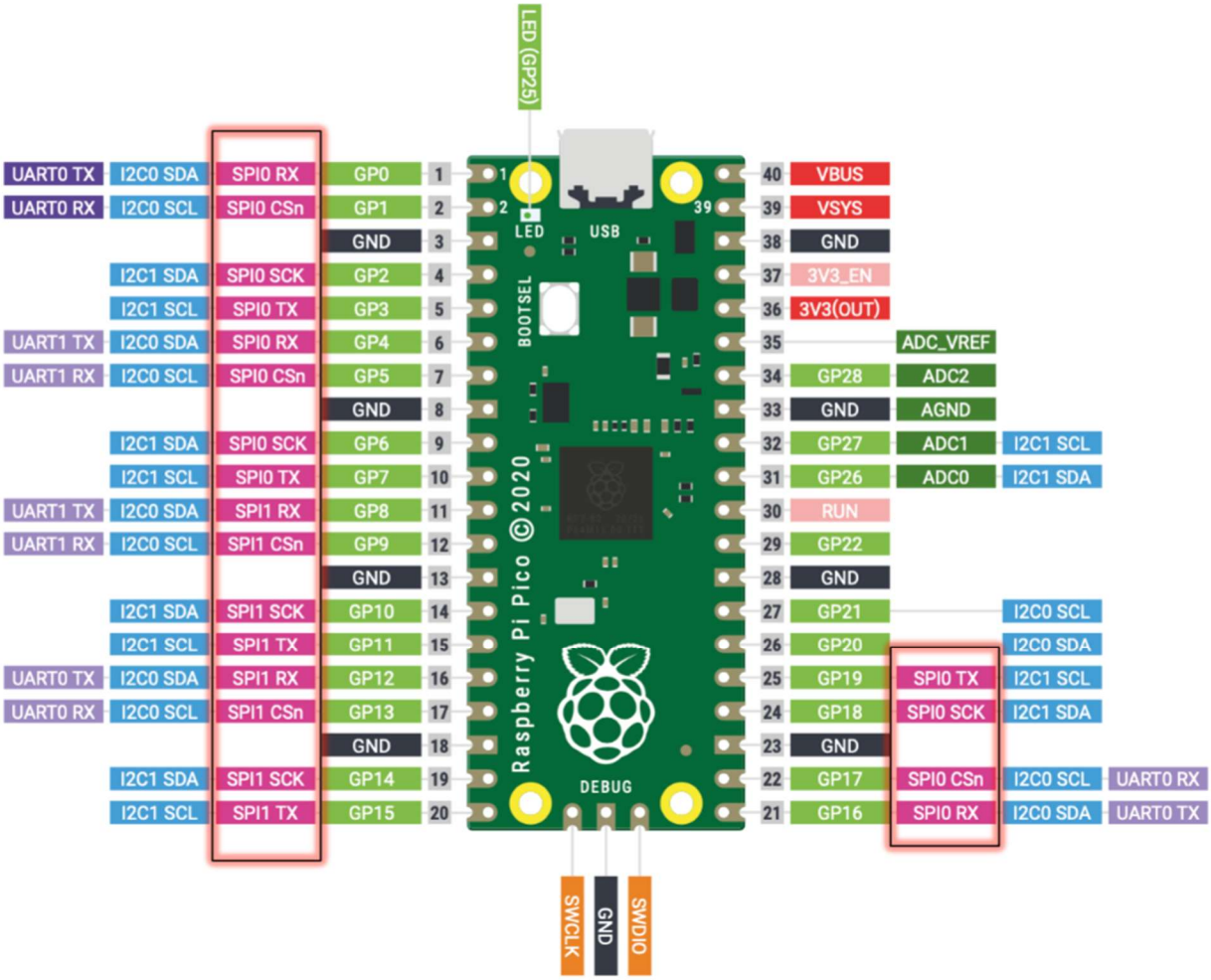


Figure 1: Raspberry Pi Pico 2 Layout

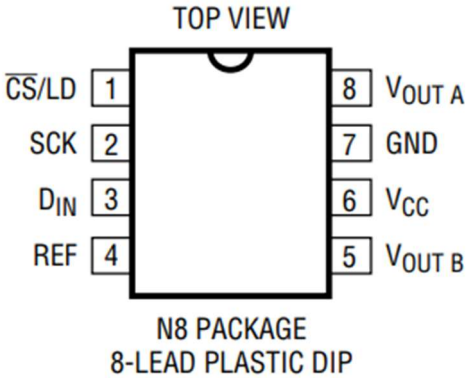


Figure 2: LTC1661 Pinout

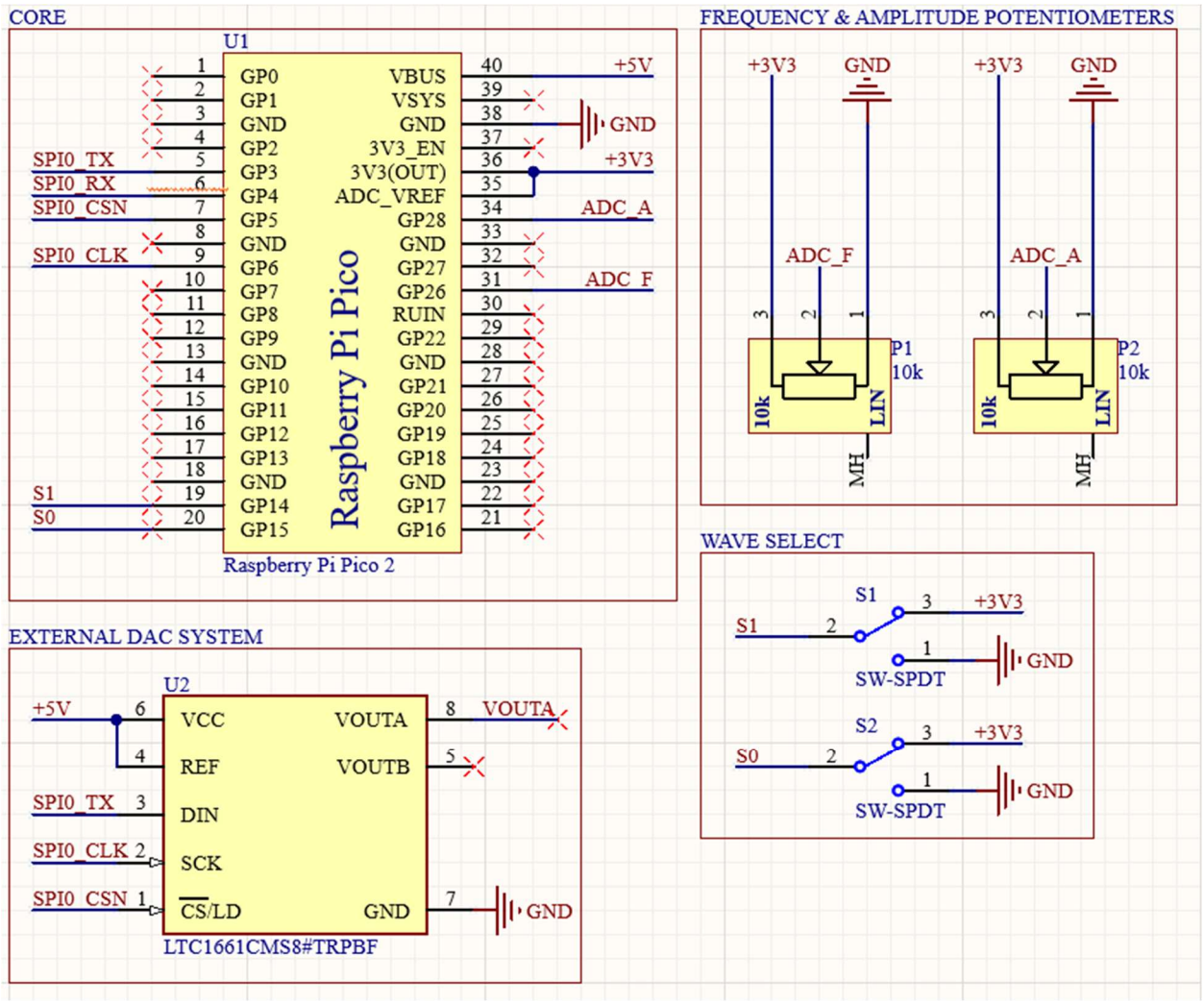


Figure 3: DAC System in Altium Designer

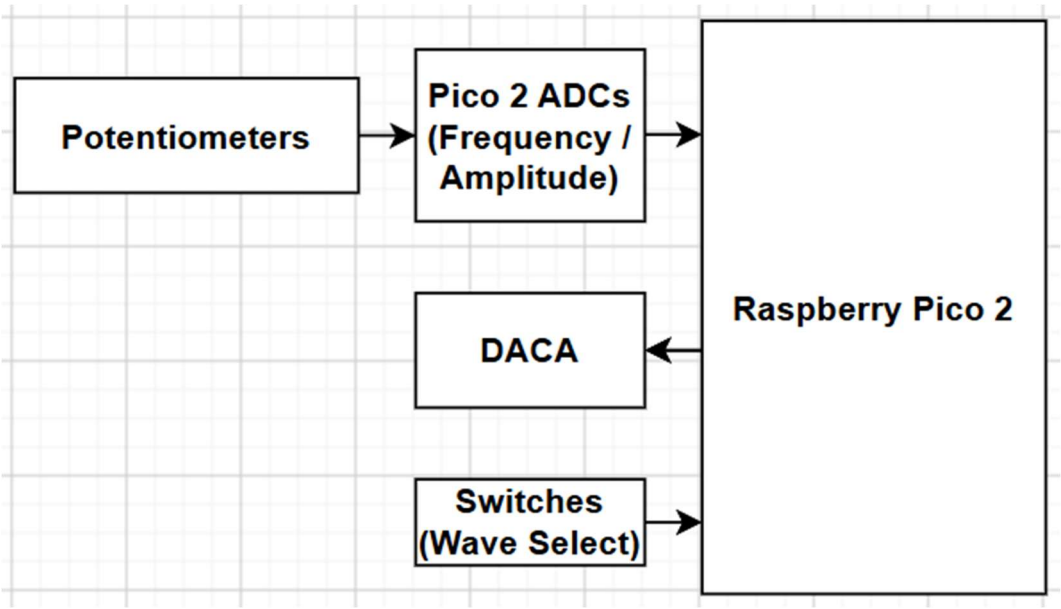


Figure 4: Flowchart of Hardware

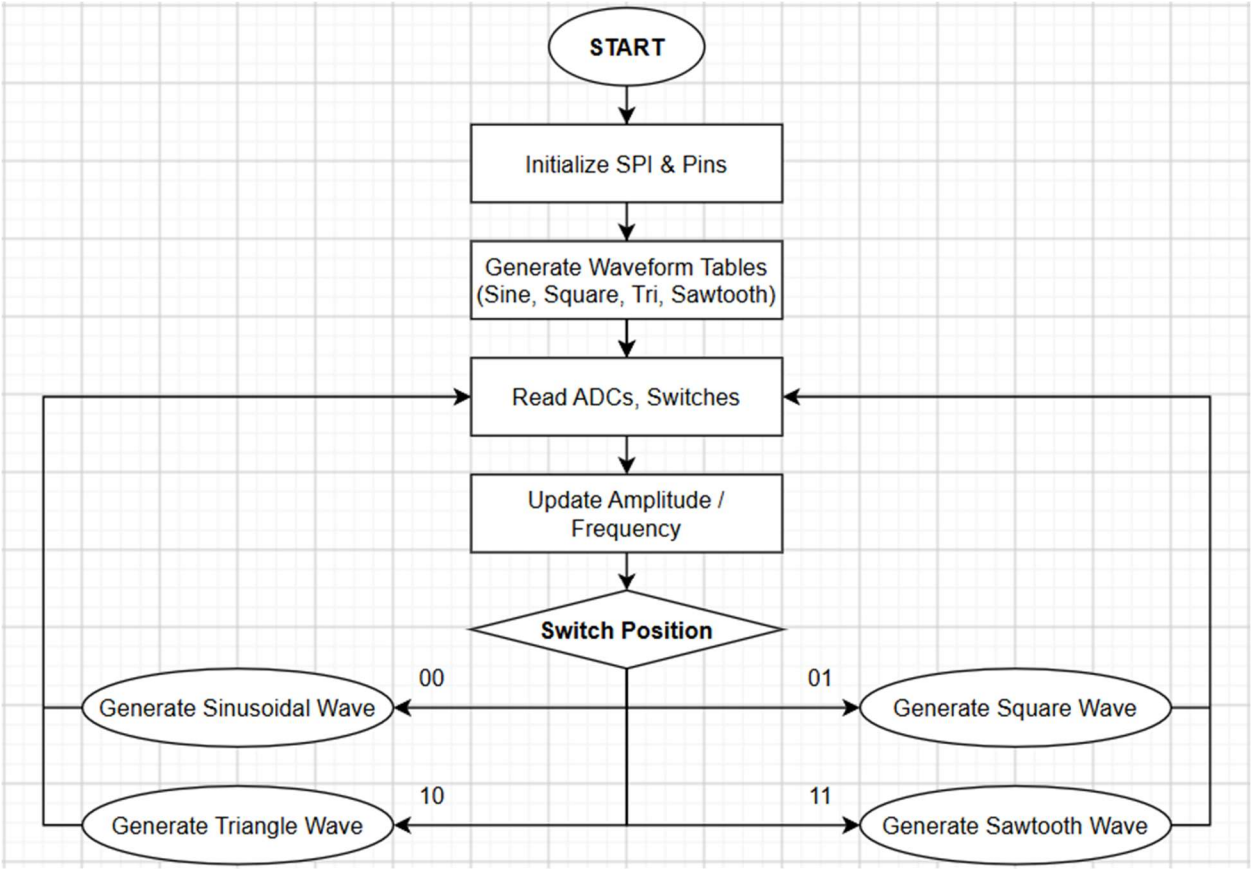


Figure 5: Flowchart of Software

Results

The four outputs are shown below at a mid-point frequency and voltage for readability and clarity on the DAD 3:

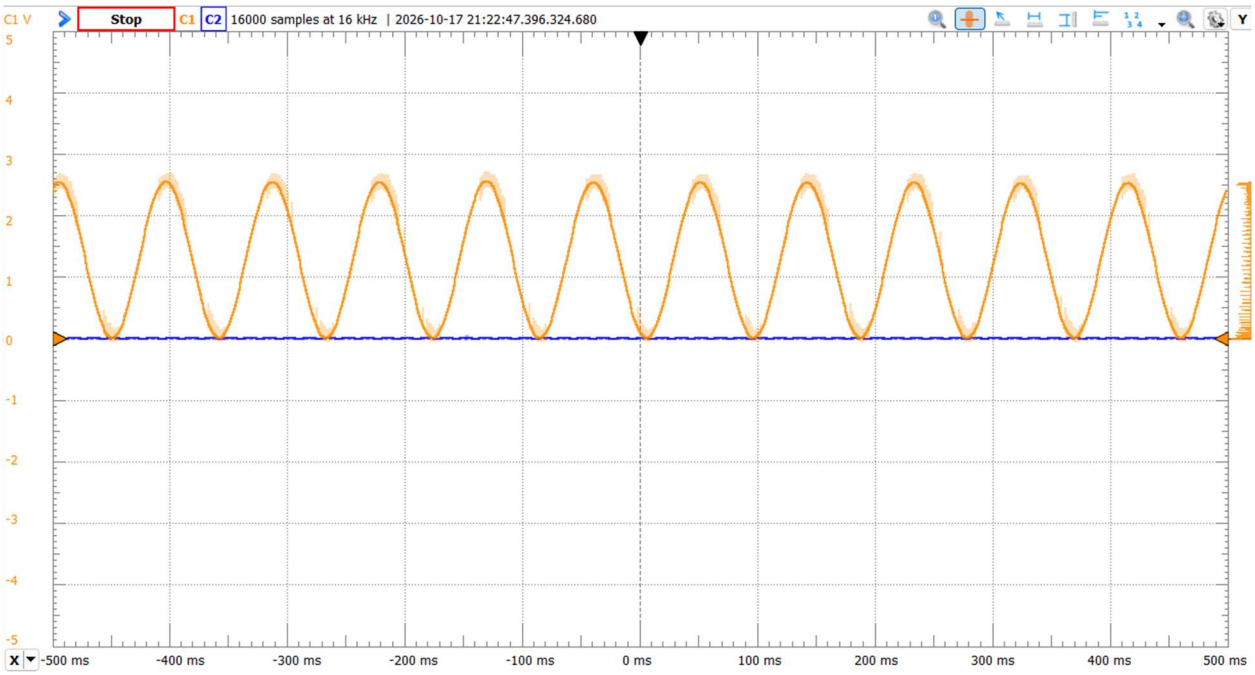


Figure 6: Sinusoidal Waveform Output

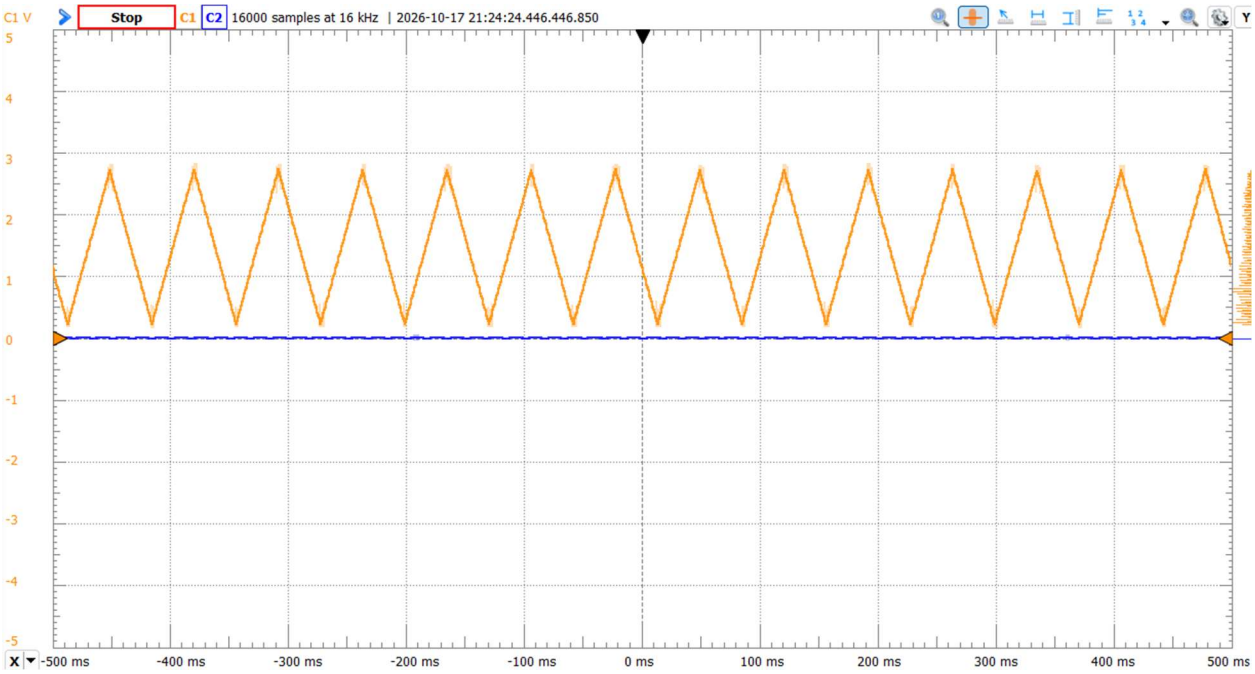


Figure 7: Triangle Waveform Output

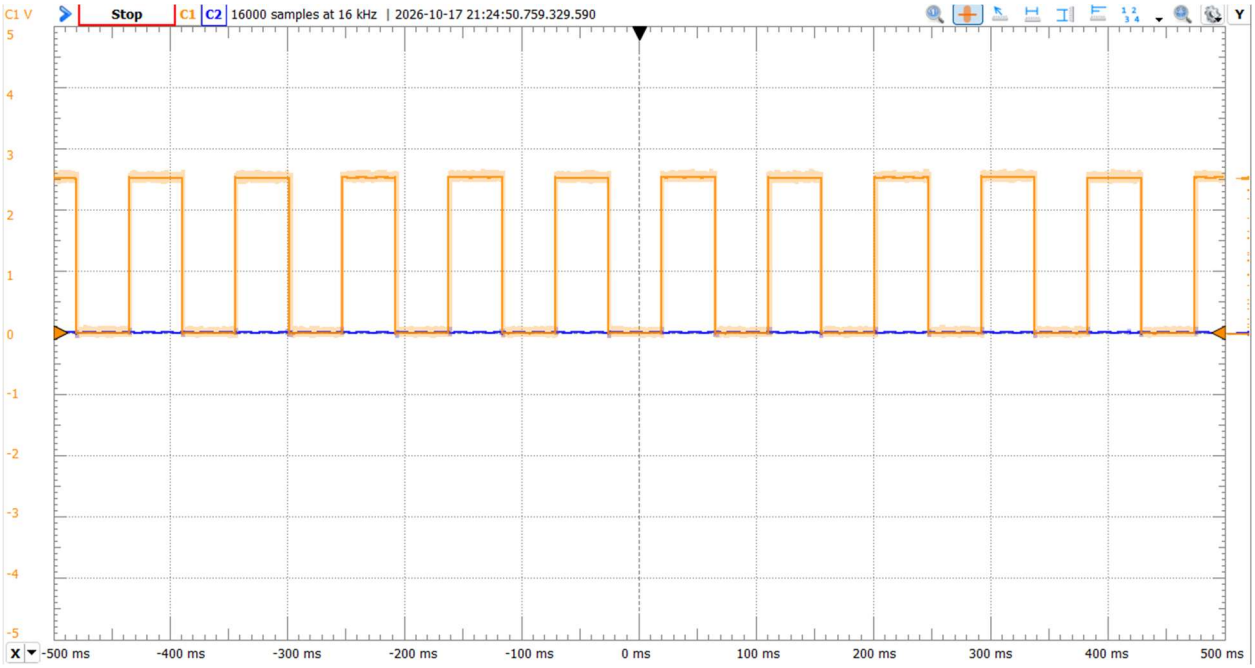


Figure 8: Square Waveform Output

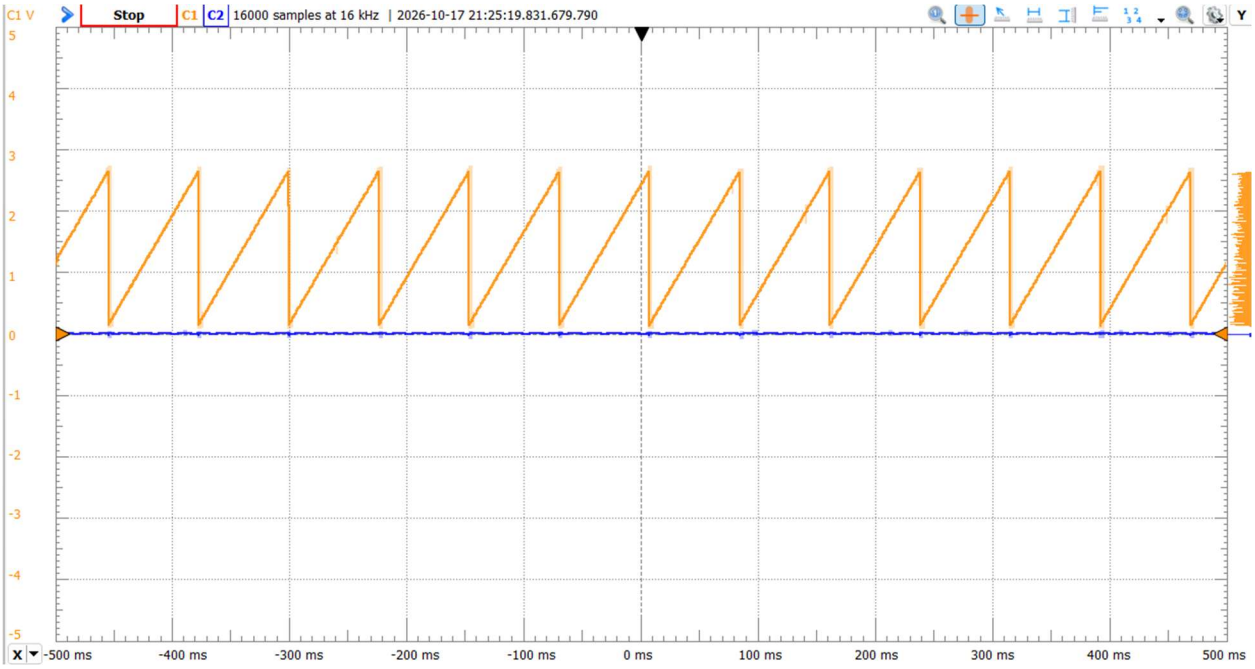


Figure 9: Sawtooth Waveform Output

As mentioned above, the frequency and amplitude are controlled through potentiometers on ADC channels 0 and 2, this is to prevent any loading, easy control from rail to rail, and for easier positioning on the board. This also helps us, as direct frequency control would not be feasible, as it is dependent on turning a voltage level to a sample period, which requires internal logic that is not attainable with a potentiometer alone.

Conclusion

This module reinforced our implementation of communication protocols to external ICs, and our familiarity with DAC systems. It also challenged us by converting voltage fluctuation into frequency control. We utilized the challenges posed by this assignment to create a high-clarity, 10-100Hz, 0-5V, 4-function waveform generator for our signal outputs.

Appendix

```
# ---------------------------------
#   LTC1661 DAC Waveform Firmware
# ---------------------------------

# Written by Evan Baesler
# October 13, 2026
# For: Raspberry Pi Pico 2

# Abstract:
# This program runs on a Raspberry Pi Pico 2 to generate four waveform
# types (sinusoidal, square, triangular, and sawtooth) with variable
# frequency and amplitude based on inputs from potentiometers. The
# waveform switching is handled by two external switches, similar to a
# multiplexer/mux. Functionally, our LTC1661 can generate any of the
# four waveforms between 10Hz to 100Hz and 0V to 5V.

## SETUP

from machine import ADC, Pin, SPI
import math
import utime

# Pins are in terms of GPIO Ports

# LEGEND:
# XP = External Potentiometer to ADC

# Waveform Frequency (XP)
# Amplitude (XP)
```

```
# SPI pin configuration for Raspberry Pi Pico
spi = SPI(
    0,
    baudrate = 6000000,
    polarity = 0,
    phase = 0,
    bits = 8,
    firstbit = SPI.MSB,
    sck = Pin(6),
    mosi = Pin(3),
    miso = Pin(4)
)

# Chip select (active-low) for the DAC
cs = Pin(5, Pin.OUT)
cs.value(1)  # DAC initially deselected

# Switch Values
switch1 = Pin(14, Pin.IN)
switch2 = Pin(15, Pin.IN)

# ADC Pins
adc_freq = ADC(28)
adc_amp = ADC(26)

# WAVEFORM TABLE MATH

NUM_SAMPLES = 110
DAC_MAX = 1023
```

```
# Generate each table once during startup
sine_table = []
square_table = []
triangle_table = []
sawtooth_table = []

for i in range(NUM_SAMPLES):
    phase = i / NUM_SAMPLES

    # Sine: 0 -> maximum -> 0
    sine_value = (1 + math.sin(2 * math.pi * phase)) / 2
    sine_table.append(int(sine_value * DAC_MAX))

    # Square: high for first half, low for second half
    square_table.append(DAC_MAX if phase < 0.5 else 0)

    # Triangle: linear rise, then linear fall
    if phase < 0.5:
        triangle_value = 2 * phase
    else:
        triangle_value = 2 * (1 - phase)

    triangle_table.append(int(triangle_value * DAC_MAX))

    # Sawtooth: linear rise, then reset
    sawtooth_table.append(int(phase * DAC_MAX))

# Store references to all four tables
waveforms = [
    sine_table, # 00
    square_table, # 01
    triangle_table, # 10
    sawtooth_table # 11
]
```

```
## END SETUP
## SUBROUTINES
# ADC (returns averaged raw counts, 0-65535)

def adc_update(samples=1):
    vfreq_total = 0
    vamp_total = 0

    for i in range(samples):
        vfreq_total += adc_freq.read_u16()
        vamp_total += adc_amp.read_u16()

    freq_raw = vfreq_total // samples
    amp_raw = vamp_total // samples

    return freq_raw, amp_raw

# Write one 10-bit sample to the LTC1661 DAC

dac_buf = bytearray(2)

def dac_write(value):
    value = max(0, min(DAC_MAX, int(value)))
    word = (0b1001 << 12) | (value << 2)

    dac_buf[0] = (word >> 8) & 0xFF
    dac_buf[1] = word & 0xFF

    cs.value(0)
    spi.write(dac_buf)
    cs.value(1)

## END OF SUBROUTINES
```

```
## PROGRAM START

while True:

    # Read potentiometers
    freq_raw, amp_raw = adc_update()

    # Map frequency potentiometer to 10-100 Hz
    frequency = 10 + (freq_raw * 100 // 65535)

    # Map amplitude potentiometer to 0-1023 DAC counts
    amplitude = DAC_MAX * amp_raw // 65535 # amp_raw = 65535 -> DAC_MAX out

    # Read switches
    s1 = switch1.value()
    s2 = switch2.value()

    # Select waveform
    wave_select = (s1 << 1) | s2
    table = waveforms[wave_select]

    # Calculate time per sample
    sample_period_us = 1000000 // (frequency * NUM_SAMPLES)
    # Sample period in us = 1 / (frequency * samples per second)

    # Output one complete waveform cycle

    # Output one complete waveform cycle
    next_sample = utime.ticks_us()

    for sample in table:
        # Scale the sample for the selected amplitude
        dac_value = sample * amplitude // DAC_MAX
```

```
        # Write to the DAC
        dac_write(dac_value)

        # Advance the target time by exactly one sample period
        next_sample = utime.ticks_add(
            next_sample, sample_period_us
        )

        # Wait only until the scheduled deadline
        remaining = utime.ticks_diff(
            next_sample, utime.ticks_us()
        )

        if remaining > 0:
            utime.sleep_us(remaining)


## PROGRAM END
```

References

ChatGPT for code debugging & optimization

Digi Key for Datasheets (https://www.analog.com/media/en/technical-documentation/data-sheets/1661fb.pdf)